ETH zürich

ETH Library

A Direct Memory Access Controller (DMAC) for Irregular Data Transfers on RISC-V Linux Systems

Working Paper

Author(s):
Benz, Thomas; Vanoni , Axel; Rogenmoser, Michael; Benini, Luca

Publication date:
2025-10-14

Permanent link:
https://doi.org/https://doi.org/10.3929/ethz-c-000787371

Rights / license:
In Copyright - Non-Commercial Use Permitted

Originally published in:
arXiv, https://doi.org/10.48550/arXiv.2510.12277

This page was generated automatically upon download from the ETH Zurich Research Collection.
For more information, please consult the Terms of use.

A Direct Memory Access Controller (DMAC) for Irregular Data Transfers on RISC-V Linux Systems

Thomas Benz[‡*], Axel Vanoni[‡*], Michael Rogenmoser[*], Luca Benini[*†]
[*]*ETH Zurich*, Zurich, Switzerland
[†]*University of Bologna*, Bologna, Italy
{tbenz,axvanoni,michaero,lbenini}@ethz.ch

arXiv:2510.12277v1 [cs.AR] 14 Oct 2025

***Abstract*—With the ever-growing heterogeneity in computing systems, driven by modern machine learning applications, pressure is increasing on memory systems to handle arbitrary and more demanding transfers efficiently. Descriptor-based direct memory access controllers (DMACs) allow such transfers to be executed by decoupling memory transfers from processing units. Classical descriptor-based DMACs are inefficient when handling arbitrary transfers of small unit sizes. Excessive descriptor size and the serialized nature of processing descriptors employed by the DMAC lead to large static overheads when setting up transfers. To tackle this inefficiency, we propose a descriptor-based DMAC optimized to efficiently handle arbitrary transfers of small unit sizes. We implement a lightweight descriptor format in an AXI4-based DMAC. We further increase performance by implementing a low-overhead speculative descriptor prefetching scheme without additional latency penalties in the case of a misprediction. Our DMAC is integrated into a 64-bit Linux-capable RISC-V SoC and emulated on a Kintex FPGA to evaluate its performance. Compared to an off-the-shelf descriptor-based DMAC IP, we achieve 1.66× less latency launching transfers, increase bus utilization up to 2.5× in an ideal memory system with 64-byte-length transfers while requiring 11% fewer lookup tables, 23% fewer flip-flops, and no block RAMs. We can extend our lead in bus utilization to 3.6× with 64-byte-length transfers in deep memory systems. We synthesized our DMAC in GlobalFoundries' GF12LP+ node, achieving a clock frequency of over 1.44 GHz while occupying only 49.5 kGE.**

***Index Terms*—DMAC, Transfer descriptor, Memory system, SoC, AMBA4 AXI**

I. Introduction

Modern computing systems are rapidly increasing in complexity and scale to combat the slowdown of Dennard scaling and to satisfy the ever-increasing need for more computing performance and memory, driven by machine learning (ML) and big data workloads [1]. Kumar et al. highlight the importance of irregular memory accesses in sparse data structures when dealing with large-scale graph applications [2]. Today's systems require high-performance interconnects with components that efficiently move the data required to supply their compute units. Using such specialized direct memory access controllers (DMACs) is a well-established method to transfer data independently of processors, thereby promising to achieve high throughput while the processor is free to perform computationally useful work [3]–[8]. With a shift towards more heterogeneous architectures, as well as smaller datatypes [1], [9], more diverse transfers are emerging, requiring greater flexibility and less overhead in the programmability of DMACs.

Highly flexible programming hardware interfaces and hardware abstraction layers for DMACs are usually based on *descriptors*, data structures stored in shared memory that hold the information of a transfer. Descriptors have multiple advantages compared to simpler register-based programming interfaces, which are widely used in embedded and microcontroller unit (MCU) applications [10]. Descriptors massively reduce the requirement for dedicated configuration memory space by storing the transfer specification in general-purpose memory segments, thus eliminating the need for configuration space replication in multicore applications to enforce atomic transfer launching [10]. Descriptors can be chained as a linked list, enabling the automatic launch of subsequent transfers. This allows multidimensional affine and fully arbitrary and irregular workloads to be processed [11]. A concrete example of a DMAC is the *LogiCORE IP DMA*, an Advanced eXtensible Interface (AXI) direct memory access (DMA) soft intellectual property (IP) by *Xilinx* [7]. It is a high-bandwidth DMAC with a descriptor-based programming interface and is designed to transfer data between a memory-mapped AXI interface and an AXI-Stream target device.

Applying this descriptor configuration model to fine-grained, irregular transfers leads to long chains of individual descriptors, requiring the DMAC unit to handle a large amount of data when executing the transfer. Excessive descriptor size degrades the throughput of such fine-grained transfers, as the DMAC may require multiple cycles to fetch the descriptors. Furthermore, larger descriptor sizes result in more significant resource utilization and power overhead required by buffering logic. *Synopsys' DesignWare* AXI DMA controller presents a parametrizable and high-performance DMAC solution, using 64-byte-long descriptors in *scatter-gather mode* [8]. Paraskevas et al. describe an efficient 32-byte-long descriptor format without prefetching capabilities [12]. In their work, descriptors are stored in dedicated pages of the core's on-chip scratchpad memory. Ma et al. describe a five-entry-long descriptor format supporting chaining for multidimensional data transfers [13]. To ensure efficient fetching of the descriptors, they are stored in a dedicated DMAC-internal *parameter RAM*.

As descriptors are usually handled in sequence [7], requesting the next descriptor once the prior is read, full DMAC

[‡] Both authors contributed equally to this research.

utilization is only reached if the described transfer is long enough to hide the latency of fetching a descriptor. This can no longer be guaranteed for fine-grained transfers in a non-ideal memory system, limiting the maximum achievable DMAC utilization for such transfers.

In this work, we tackle both of these issues by introducing a DMAC with a minimal descriptor format, as well as a low-overhead prefetching mechanism; our contributions are:

1) A lightweight, minimal, and efficient descriptor format holding only the essential information required to describe a transfer. Our format supports chaining and provides a mechanism to track transfer completion, increasing DMAC utilization by 3.9× for 64-byte transfers compared to the *LogiCORE IP DMA* [7].
2) Implementing our descriptor format, as well as speculative descriptor prefetching, together with an existing Advanced Microcontroller Bus Architecture (AMBA) AXI DMA engine [14], creating a fully parametrizable, synthesizable, and technology-independent DMAC.
3) Evaluating our DMAC out of context (OOC) regarding its performance, area requirements, and timing in a 12 nm node. The DMAC can achieve near-ideal performance while exceeding clock frequencies of 1.44 GHz and requiring only 49.5 kGE.
4) Integrating the resulting DMAC into a 64-bit RISC-V SoC [15] using general-purpose DDR3 memory to store our descriptors. We achieve an improvement in terms of latency by 1.66× compared to the *LogiCORE IP DMA* AXI DMA [7], while requiring 11% fewer lookup tables, 23% fewer flip-flops, and no block RAMs.

II. Architecture

With efficient data transfer being an essential requirement for ML workloads, we make use of the low-level DMA engine presented by Kurth et al. in [14]. This fully open-source DMA engine directly interfaces with an AMBA AXI memory system and is capable of asymptotically utilizing the available bandwidth. Furthermore, the DMA engine in [14] is optimized for low area utilization, low transfer launch latencies, and high clock frequencies; however, it does not directly provide a programming interface to the system.

In the following section, we describe a descriptor-based programming interface called *DMA frontend*. The *DMA frontend* and *DMA engine* as *backend* together form the DMAC, as shown in Figure 1.

A. DMA Frontend Design

To configure a DMA transfer, the DMA frontend exposes a memory-mapped configuration and status register (CSR), which accepts an address pointing to a DMA transfer descriptor in shared memory, described in Section II-B. Once this pointer is written into the CSR, the frontend requests the descriptor from memory through the read channel of an AXI manager port, shown as the request logic in Figure 1. This manager port is configurable in both AXI address width, allowing descriptors to be located in any memory location,

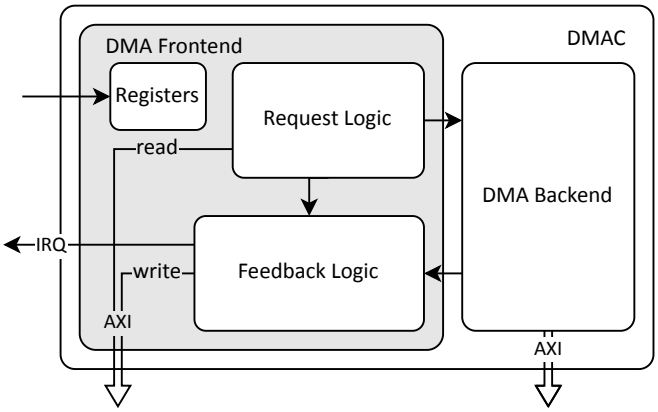


Fig. 1. Overview of the DMAC, containing request logic with internal registers for configuration and read logic to fetch the descriptor, and feedback logic to update the system once the DMA backend completes the transfer.

and AXI data width, ranging from 16 bits to 512 bits. Using this port, the frontend retrieves the necessary information for a generic linear memory transfer: source address pointer, destination address pointer, transfer length, and configuration.

Once fetched, the frontend forwards the information to the DMA backend, which executes the transfer. To improve performance, both the CSR and the connection to the backend implement a queue. This allows multiple transfers to be enqueued, maximizing the utilization of the backend.

Once the DMA backend has completed a transfer, the frontend reports completion back to the system, shown as the feedback logic in Figure 1. For each transfer, the corresponding descriptor is modified to indicate its completion, and an interrupt is signaled if configured.

B. DMAC Transfer Descriptor

Our DMAC descriptor contains the information necessary to fully describe a linear memory transfer: a 64-bit source and destination address, and the length of the transfer. The transfer length is stored in an unsigned 32-bit field, allowing individual transfers of up to 4 GiB in size. Longer transfers can easily be achieved by chaining together multiple descriptors.

A *config* field in the descriptor holds configuration information for both the DMA front- and backend. For the former, different interrupt request (IRQ) options can be set, while for the latter, various AXI-related parameters are configurable. A complete descriptor structure can be found in Listing 1.

Apart from information describing the transfer, the descriptor contains a pointer to the next descriptor to be processed, enabling *descriptor chaining*. This allows our DMAC to process a *linked list* or *chain of descriptors* in memory without involving the central processing unit (CPU). The last descriptor in a chain carries all ones (equals to -1) in the *next* field; we call this value *end-of-chain*. This value was chosen as no descriptor can fit at the corresponding address. Descriptor chaining allows the construction of arbitrary and irregular transfers from simple linear transfers.

When designing the descriptor format, we minimized its size while keeping it a multiple of the AXI bus width. The former has two benefits; it not only reduces the required bandwidth of the memory subsystem when storing and fetching descriptors, but also the overall memory footprint to describe a given transfer. The latter allows us to fetch the 256-bit descriptors and chains thereof without losing utilization in memory systems with widths up to 256 bits. In systems featuring a 512-bit infrastructure, such as a wide range of *Xilinx Zynq UltraScale+ MPSoCs*, two full descriptors could be fetched in one cycle.

Listing 1. Descriptor Layout

```
struct descriptor {
  u32 length;
  u32 config;
  u64 next;
  u64 source;
  u64 destination;
}
```

To compare, the *LogiCORE IP DMA* [7] uses a descriptor format of thirteen 32-bit words or 416 bits, of which usually only 256 bits are read. Its AXI manager interface used to fetch descriptors is limited to a data width of 32 bits, leading to a descriptor read latency of at least eight to thirteen cycles. In contrast, our DMAC may read a descriptor in four cycles in a comparable 64-bit system.

C. Speculative Descriptor Prefetching

To compensate for memory latency, we employ *speculative descriptor prefetching*. Once a descriptor address is written to the CSR, we not only request the first descriptor over the frontend's manager interface but send up to a configurable amount of requests with sequential addresses. The number of descriptors speculatively requested is configured using the *prefetching* compile-time parameter, zero deactivating the prefetching logic, as can be seen in Table I.

Once a descriptor arrives at the DMA frontend, we compare the *next* field of this descriptor with the speculatively requested address. On a match, the speculative address is committed and one speculation slot is freed up. Should a misprediction occur, we discard all descriptor addresses in the *speculation slots* and start to fetch from the correct *next* address while ignoring the incoming data that was mispredicted.

Care was taken not to introduce any latency in the case of mispredictions: Assuming there is space in the *speculation slots*, the proper request is issued over the AXI manager interface in the same cycle the DMA frontend receives the *next* field. This is the same latency we observe in the case of prefetching disabled. Thus, the only performance degradation that may occur is caused by minimal additional contention in the memory system due to fetching data that is directly discarded.

D. SoC integration

Heterogeneous systems often rely on a high-performance 64-bit memory system due to compatibility with the central host processor. While parametrizable, our implementation configures the DMAC for such a memory system, using 64 bits both for address and data width of the AXI bus in accordance with the *CVA6 RISC-V SoC* [15] we integrate our DMAC into. An overview of the resulting system can be seen in Figure 2: The two manager interfaces of our DMAC, as well as the subordinate configuration interface, are connected to the memory system of the system on chip (SoC).

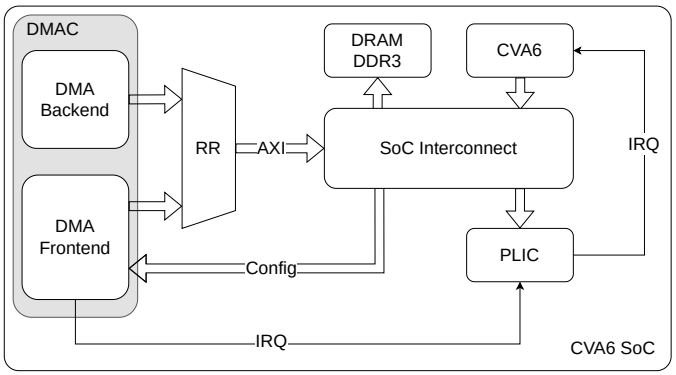


Fig. 2. Integration of our DMAC into the CVA6 SoC. The two manager interfaces, after arbitration, as well as the subordinate configuration port of the DMAC, are connected to the SoC's interconnect. The IRQ line is connected to the platform's PLIC.

We occupy one new IRQ channel at the systems' Platform-Level Interrupt Controller (PLIC), which is used to signal transfer completion when configured. For lightweight in-system progress reporting, we repurpose a transfer descriptor by overwriting its first 8 bytes with *all ones* after the transfer is completed. This allows us to forego raising an interrupt after each linear transfer is completed, thus making interrupt notification optional.

E. Linux Driver

To ensure simple integration into existing environments, we provide a sample *Linux* driver with an accompanying device-tree file. The DMA subsystem of the Linux kernel exposes a broad application programming interface (API) [16], of which we implement the *memcpy* interface.

For a DMA client to request a data transfer, the application requests the driver to prepare the *memcpy* transfer. This is done by allocating one or more chained descriptors and populating *source*, *destination*, *length*, and *config* fields. Should a transfer consist of more than one descriptor, then only the last has IRQ signaling enabled.

As a second step, the client commits to specific transfers, which results in the driver chaining them in a FIFO fashion to a new chain.

Third, the client requests to submit all committed transfers to the hardware. The driver checks whether less than the maximum number of allowed chains are already running on the DMAC; if so, it schedules the new chain with a write to the DMAC's CSR, otherwise, the transfers are stored to be scheduled later.

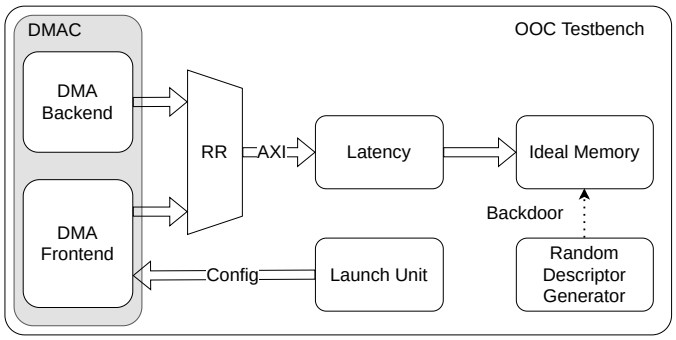


Fig. 3. The OOC testbench setup; the DMAC has its two AXI manager interfaces connected to a fair round-robin arbiter (RR), which in turn is connected to a latency-configurable memory system. Descriptors are loaded into the memory using backdoor access and are launched via the DMAC's subordinate configuration interface.

Finally, on transfer completion, the DMAC raises an IRQ. This leads to a call of the *interrupt handler*, which schedules any completion callbacks the client has registered, updates the number of active chains if the transfer was the last of a chain, and schedules stored transfers.

III. Results

We evaluate our DMAC with our optimized descriptor format out of context (OOC) and in-system. For the OOC evaluation, we attach our DMAC to a configurable memory system to assess its performance, as can be seen in Figure 3. We then present area and timing results from synthesizing our controller out-of-context using a 12 nm FinFET node.

We then show both performance and implementation results of our DMAC integrated into a 64-bit RISC-V CVA6 SoC [15] emulated on a *Diligent Genesys 2* field programmable gate array (FPGA) [17].

A. Out-of-context Results

To evaluate the standalone performance of our DMAC, we created a testbench environment consisting of a *latency-configurable* memory system and a *launch unit* to set up and execute random streams of descriptors. To simulate a real system, both of our DMAC's AXI manager ports are connected to the same memory system using a fair round-robin arbiter (RR), as shown in Figure 3. To stay aligned with our target CVA6 SoC, we set the address and data width of our OOC testbench to 64 bits.

The randomness of the descriptions can be closely controlled, allowing us to emulate different transfer characteristics. The corresponding descriptors are immediately preloaded into our simulation memory using a backdoor, while the actual launch of the transfers is controlled using our DMAC's CSR interface.

The bus utilization is measured at the DMA backend's AXI *manager* interface; only *useful* payload traffic contributes to utilization. We only report *steady state* bus utilization suppressing any possible cold-start phenomena.

TABLE I
THE COMPILE-TIME PARAMETERS USED IN OUR OOC EXPERIMENTS.

Configuration	Descriptors In-flight	Prefetching
LogiCORE IP DMA [7]	4	*N.A.*
base	4	Disabled (0)
speculation	4	4
scaled	24	24

In our analysis, we assessed three distinct memory system configurations reflecting different use cases:

1) *Ideal Memory:* We configure our simulation memory to have one cycle latency emulating an SRAM-based main memory.
2) *DDR3 Main Memory:* Replicating the conditions found on the *Diligent Genesys 2* FPGA [17] when accessing DDR3 off-chip memory, we include a configuration with *thirteen* cycles latency.
3) *Ultra-deep Memory:* Representing a large network on chip (NoC) system found in a modern SoC, we include a configuration with a latency of *one hundred* cycles.

To ensure a fair comparison against the *LogiCORE IP DMA*, we include a *base* configuration closely matching the *LogiCORE IP DMA*'s default configuration. In our evaluation, we included two additional configurations; one enabling *speculation* while closely resembling the *base* configuration and a *scaled* configuration setting both the number of *descriptors in-flight* and the *prefetching* to *24*. We summarize the respective parameter configurations in Table I.

As access to main memory is shared between the DMA frontend and DMA backend, the bus utilization, as defined above, cannot reach 100%. The transfer of the payload will be interrupted by descriptor transfers, limiting the *ideal bus utilization*, $\bar{u}$ – see Equation (1) – where n is the transfer size in byte.

$$\bar{u} = \frac{n}{n + 32} \tag{1}$$

Descriptor misprediction, in the case of speculative prefetching enabled, further limits the ideal utilization, as it inflates the number of additional bytes fetched by the DMA frontend per transfer.

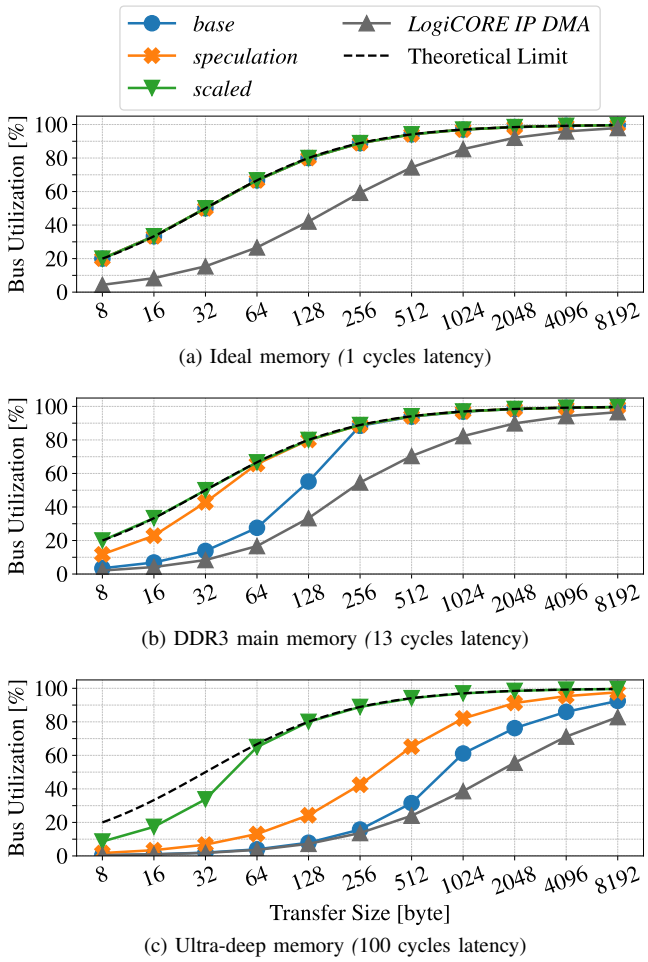
In very shallow or ideal memory systems, our *base* configuration already achieves ideal steady-state utilization for any bus-aligned transfer size, as shown in Figure 4a. At transfer sizes of 64 B – a typical cache line size in many memory architectures – we improve the utilization by 2.5× compared to the *LogiCORE IP DMA*.

When using the *Genesys 2 DDR3* latency configuration, we achieve ideal steady-state utilization at 256 B without and 64 B with prefetching enabled, as can be seen in Figure 4b. This increases the utilization by up to 1.7× and 3.9×, respectively, compared to the *LogiCORE IP DMA*.

Finally, we show that our DMAC can be configured to still achieve near-ideal steady state utilization even in ultra-deep

TABLE II
AREA REQUIREMENTS AT THE MAXIMUM CLOCK FREQUENCY OF THE DMAC AND ITS MAIN SUB-COMPONENTS; THE DMA FRONTEND AND THE DMA BACKEND. CLOCK FREQUENCIES ARE ACHIEVED IN TYPICAL CONDITIONS.

Configuration	DMA Frontend	DMA Backend	Total DMAC	Achievable Clock Frequency
base	25.8 kGE	15.4 kGE	41.2 kGE	1.71 GHz
speculation	34.8 kGE	14.7 kGE	49.5 kGE	1.44 GHz
scaled	151.1 kGE	37.3 kGE	188.4 kGE	1.23 GHz



(a) Ideal memory (1 cycles latency)

(b) DDR3 main memory (13 cycles latency)

(c) Ultra-deep memory (100 cycles latency)

Fig. 4. DMAC *steady-state* bus utilization given a prefetch hit rate of 100% in memory systems featuring various latencies.

memory systems. As can be seen in Figure 4c, the *scaled* configuration achieves ideal utilization starting from 128 B.

Varying prefetching hit rates of 75 % to 0 %, our achievable increase in bus utilization compared to the *LogiCORE IP DMA* still ranges from 1.65 × to 3.1 × at 64 B, see Figure 5.

We evaluate the timing and resource requirements of our DMAC in the various configurations presented in Table I by synthesizing our work in *GlobalFoundries' GF12LP+ FinFET* technology using *Synopsys' Design Compiler NXT* in *topological* mode. All results are presented in the typical corner of the library at 25°C at 0.8 V, in Table II.

Our *base* configuration requires an area of 41.2 kGE, achieving a maximum clock frequency of 1.71 GHz. Enabling

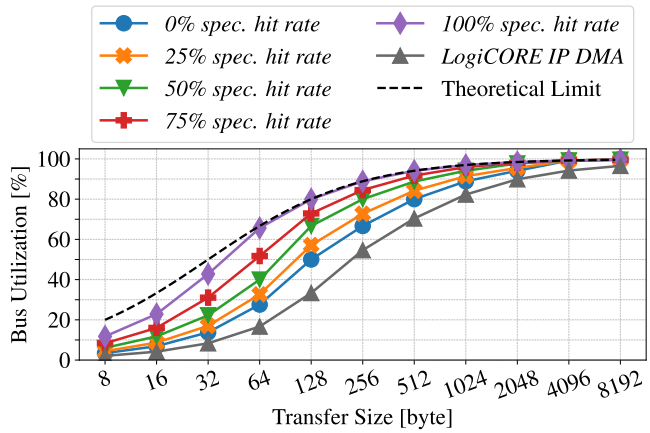


Fig. 5. DMAC steady-state bus utilization in the case of the DDR3 main memory with speculation misses; *speculation* configuration.

prefetching adds 8.3 kGE while reducing the achievable maximum clock frequency to 1.44 GHz.

We synthesized our design in numerous configurations, creating a model of the circuit area as a function of the parameters in Table I. The design's area in *kGE* is described by: $A = 20.30 + 5.28d + 1.94s$, where d denotes the number of descriptors in flight and s the number of speculatively launched descriptors. The total area is linear in d and s, allowing the hardware to be easily scaled to larger sizes.

The *scaled* configuration requires a total of 188.4 kGE achieving 1.23 GHz. Comparing these numbers to CVA, we find the DMAC area to be less than 10 % of the core's area while achieving similar clock speeds, confirming the scalability of our controller.

B. In-system Results

To evaluate the required resources on a FPGA, we synthesized the CVA6-SoC [15] with the various configurations of our DMAC integrated. Synthesis was done using *Vivado 2019.2* targeting the *Genesys 2* board, which features a *Kintex 7* FPGA from *Xilinx*.

In the *base* configuration, the footprint of the DMAC is 2610 lookup tables (LUTs) and 3090 flip-flops (FFs), while the entire SoC occupies 79142 LUTs and 58086 FFs, see Table III. This puts the base configuration at 3.3% of total LUT usage and 5.3% of total FF usage, and is a reduction of 6.25% LUT and 39.8% FF utilization compared to the *LogiCORE IP DMA*.

Compared to the *base* configuration, the *speculation* configuration uses 27% more FFs, but reduces the number of LUTs

TABLE III
FPGA RESOURCE REQUIREMENTS OF THE DMAC AT 200 MHZ.

Configuration	LUTs	FFs
base	2610	3090
speculation	2480	3935
scaled	6764	11353
LogiCORE IP DMA [7]	*2784*	*5133*

TABLE IV
DMAC LATENCIES BETWEEN VARIOUS EVENTS AND MEMORY SYSTEMS FOR THE *scaled* CONFIGURATION

Metric		*LogiCORE IP DMA* DMA [7]	*scaled*
`i-rf`		10	3
`rf-rb`	1 cycle latency	22	8
	13 cycles latency	48	32
	100 cycles latency	222	206
`r-w`		1	1

by 5%. The scaled configuration increases resource utilization further, requiring 2.59× as many LUTs and 3.67× as many FFs as the *base* configuration.

We use our latency-configurable memory system presented in Section III-A, which we integrate into the upstream CVA6-SoC to measure the following three different latencies:

- `i-rf`: the CPU issuing a write to the DMA frontend issuing a read request
- `rf-rb`: between the issue of the read request from DMA frontend and the DMA backend
- `r-w`: the latency between the DMA engine reading and writing the same data

As can be seen in Table IV, we achieve three cycles of latency for `i-rf`, an improvement of 3.33× over the *LogiCORE IP DMA*. For `rf-rb`, we achieve a latency of eight cycles in ideal memory, 32 cycles with a memory latency of 13, and 206 cycles in the case of 100 cycles of latency. This results in an improvement of 2.75×, 1.5×, and 1.08×, respectively. Latencies for `r-w` are equal at one cycle for both our DMAC and the *LogiCORE IP DMA*.

IV. CONCLUSION

In this work, we present a scalable, platform-independent, synthesizable DMAC for fast and efficient data transfers in AXI-based systems. Compared to a competing solution, we achieve 1.66× less latency, increasing bus utilization by up to 2.5× in an ideal memory system with 64-byte transfers, overall requiring 11% fewer LUTs and 23% fewer FFs without requiring any block RAMs. In deep memory systems, we show an even more significant increase in the utilization of 3.6× with 64-byte transfers.

We show the utility of our DMAC in a 64-bit Linux-capable environment by implementing it into an SoC based around CVA6 and present a working driver for Linux. The resulting DMAC is available fully open-source.[1]

[1] `https://github.com/pulp-platform/iDMA`

REFERENCES

[1] J. Frazelle, "Chip Measuring Contest: The benefits of purpose-built chips," *Queue*, vol. 19, no. 5, pp. 5–21, Oct. 2021. [Online]. Available: https://dl.acm.org/doi/10.1145/3494834.3501254

[2] M. Kumar, M. Serrano, J. Moreira, P. Pattnaik, W. P. Horn, J. Jann, and G. Tanase, "Efficient implementation of scatter-gather operations for large scale graph analytics," in *2016 IEEE High Performance Extreme Computing Conference (HPEC)*, Sep. 2016, pp. 1–7.

[3] W. Su, L. Wang, M. Su, and S. Liu, "A Processor-DMA-Based Memory Copy Hardware Accelerator," in *2011 IEEE Sixth International Conference on Networking, Architecture, and Storage*, Jul. 2011, pp. 225–229.

[4] G. Ma and H. He, "Design and implementation of an advanced DMA controller on AMBA-based SoC," in *2009 IEEE 8th International Conference on ASIC*, Oct. 2009, pp. 419–422, iSSN: 2162-755X.

[5] D. Comisky, S. Agarwala, and C. Fuoco, "A scalable high-performance DMA architecture for DSP applications," in *Proceedings 2000 International Conference on Computer Design*, Sep. 2000, pp. 414–419, iSSN: 1063-6404.

[6] D. Chen, A. G. Gara, M. E. Giampapa, P. Heidelberger, B. Steinmacher-Burow, and P. Vranas, "DMA engine for repeating communication patterns," US Patent US7 802 025B2, Sep., 2010. [Online]. Available: https://patents.google.com/patent/US7802025B2/en

[7] AMD Xilinx, "AXI DMA v7.1 LogiCORE IP Product Guide," 2022.

[8] Synopsys, "DesignWare IP Solutions for AMBA - AXI DMA Controller." [Online]. Available: https://www.synopsys.com/dw/ipdir.php?ds=amba_axi_dma

[9] M. Jang, J. Kim, J. Kim, and S. Kim, "ENCORE Compression: Exploiting Narrow-width Values for Quantized Deep Neural Networks," in *2022 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, Mar. 2022, pp. 1503–1508, iSSN: 1558-1101.

[10] D. Rossi, I. Loi, G. Haugou, and L. Benini, "Ultra-low-latency lightweight DMA for tightly coupled multi-core clusters," in *Proceedings of the 11th ACM Conference on Computing Frontiers*, ser. CF '14. New York, NY, USA: Association for Computing Machinery, May 2014, pp. 1–10. [Online]. Available: https://doi.org/10.1145/2597917.2597922

[11] J. Fjeldtvedt and M. Orlandić, "CubeDMA – Optimizing three-dimensional DMA transfers for hyperspectral imaging applications," *Microprocessors and Microsystems*, vol. 65, pp. 23–36, Mar. 2019. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S014193311830228X

[12] K. Paraskevas, N. Chrysos, V. Papaefstathiou, P. Xirouchakis, P. Peristerakis, M. Giannioudis, and M. Katevenis, "Virtualized Multi-Channel RDMAwith Software-Defined Scheduling," *Procedia Computer Science*, vol. 136, pp. 82–90, Jan. 2018. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S187705091831545X

[13] S. Ma, L. Huang, Y. Lei, Y. Guo, and Z. Wang, "An Efficient Direct Memory Access (DMA) Controller for Scientific Computing Accelerators," in *2019 IEEE International Symposium on Circuits and Systems (ISCAS)*, May 2019, pp. 1–5, iSSN: 2158-1525.

[14] A. Kurth, W. Rönninger, T. Benz, M. Cavalcante, F. Schuiki, F. Zaruba, and L. Benini, "An Open-Source Platform for High-Performance Non-Coherent On-Chip Communication," *IEEE Transactions on Computers*, vol. 71, no. 8, pp. 1794–1809, Aug. 2022, conference Name: IEEE Transactions on Computers.

[15] F. Zaruba and L. Benini, "The Cost of Application-Class Processing: Energy and Performance Analysis of a Linux-Ready 1.7-GHz 64-Bit RISC-V Core in 22-nm FDSOI Technology," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 27, no. 11, pp. 2629–2640, Nov. 2019. [Online]. Available: https://ieeexplore.ieee.org/document/8777130/

[16] "The Linux Kernel documentation — The Linux Kernel documentation." [Online]. Available: https://www.kernel.org/doc/html/latest/index.html

[17] "Genesys 2 Reference Manual - Digilent Reference." [Online]. Available: https://digilent.com/reference/programmable-logic/genesys-2/reference-manual